

Week-8-s-2520080033

Session-15

Task-1

TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
root@Shamikahasini:~# mkdir ~/redirection_project
root@Shamikahasini:~# cd ~/redirection_project
root@Shamikahasini:~/redirection_project# pwd
~/redirection_project
root@Shamikahasini:~/redirection_project# nano combined.c
root@Shamikahasini:~/redirection_project# gcc combined.c -o combined
root@Shamikahasini:~/redirection_project# ./combined
This is a STDOUT message.
This is a STDERR message.
root@Shamikahasini:~/redirection_project# ./combined > combined_output.txt 2>&1
root@Shamikahasini:~/redirection_project# cat combined_output.txt
This is a STDERR message.
This is a STDOUT message.
root@Shamikahasini:~/redirection_project# nano duplicate.c
root@Shamikahasini:~/redirection_project# gcc duplicate.c -o duplicate
root@Shamikahasini:~/redirection_project# ./duplicate
STDOUT restored successfully.
root@Shamikahasini:~/redirection_project# cat duplicate_output.txt
This message is redirected using dup2().
File descriptor duplication successful.
root@Shamikahasini:~/redirection_project# nano merge.c
root@Shamikahasini:~/redirection_project# gcc merge.c -o merge
root@Shamikahasini:~/redirection_project# ./merge
Message 1: STDOUT
Message 2: STDERR
Message 3: STDOUT
Message 4: STDERR
root@Shamikahasini:~/redirection_project# ./merge > merged.txt 2>&1
root@Shamikahasini:~/redirection_project# cat merged.txt
Message 1: STDOUT
Message 2: STDERR
Message 3: STDOUT
Message 4: STDERR
root@Shamikahasini:~/redirection_project# nano ordering.c
root@Shamikahasini:~/redirection_project# nano ordering.c
root@Shamikahasini:~/redirection_project# gcc ordering.c -o ordering
root@Shamikahasini:~/redirection_project# ./ordering > ordering.txt 2>&1
root@Shamikahasini:~/redirection_project# cat ordering.txt
Order 1: STDOUT
Order 2: STDERR
Order 3: STDOUT
Order 4: STDERR
root@Shamikahasini:~/redirection_project# grep -n "Order" ordering.txt
1:Order 1: STDOUT
root@Shamikahasini:~/redirection_project# grep -n "Order" ordering.txt
1:Order 1: STDOUT
2:Order 2: STDERR
3:Order 3: STDOUT
4:Order 4: STDERR
root@Shamikahasini:~/redirection_project# nano edge.c
root@Shamikahasini:~/redirection_project# gcc edge.c -o edge
root@Shamikahasini:~/redirection_project# ./edge
=== Edge Case Testing ===
Error opening file: No such file or directory
Testing empty input file.
root@Shamikahasini:~/redirection_project# touch empty.txt
root@Shamikahasini:~/redirection_project# ls -l empty.txt
-rw-r--r-- 1 root root 0 Sep 24 09:20 empty.txt
root@Shamikahasini:~/redirection_project# touch readonly.txt
root@Shamikahasini:~/redirection_project# chmod 000 readonly.txt
root@Shamikahasini:~/redirection_project# ls -l readonly.txt
----- 1 root root 0 Sep 24 09:21 readonly.txt
root@Shamikahasini:~/redirection_project# chmod 644 readonly.txt
root@Shamikahasini:~/redirection_project# ls -l readonly.txt
-rw-r--r-- 1 root root 0 Sep 24 09:21 readonly.txt
root@Shamikahasini:~/redirection_project# nano integrated.c
root@Shamikahasini:~/redirection_project# gcc -Wall -Wextra integrated.c -o integrated
root@Shamikahasini:~/redirection_project# ./integrated
=====
INTEGRATED FILE DESCRIPTOR TEST
=====

[1] STDOUT TEST
Standard output is working.
[2] STDERR TEST

[3] ORDER TEST
STDERR message after STDOUT.

[4] DUPLICATE DESCRIPTOR TEST

[5] STDOUT RESTORED
Standard output is restored successfully.

[6] INTEGRATED TEST COMPLETE
root@Shamikahasini:~/redirection_project# ./integrated > final_output.txt 2>&1
root@Shamikahasini:~/redirection_project# cat final_output.txt
=====
```

```

root@Shamikahasini:~/redirection_project# cat final_output.txt
=====
INTEGRATED FILE DESCRIPTOR TEST
=====

[1] STDOUT TEST
Standard output is working.
[2] STDERR TEST

[3] ORDER TEST
STDERR message after STDOUT.

[5] STDOUT RESTORED
Standard output is restored successfully.

[6] INTEGRATED TEST COMPLETE
root@Shamikahasini:~/redirection_project# echo $?
0
root@Shamikahasini:~/redirection_project# ls
combined    combined_output.txt  duplicate.c          edge          empty.txt          integrated    integrated_
combined.c  duplicate            duplicate_output.txt edge.c        final_output.txt   integrated.c  merge
root@Shamikahasini:~/redirection_project#

```

Observation:

The program successfully parsed the combined redirection command and redirected both **standard output (stdout)** and **standard error (stderr)** to the same output file using `dup2()`. The child process executed the given command using `execvp()`, and the parent process waited for its completion using `waitpid()`. The program also checked the child exit status and closed the file descriptors successfully. Different commands were tested to verify combined redirection, output ordering, invalid commands, and missing output filenames, demonstrating handling of different edge cases.

Task-2

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>

#define MAX_INPUT 500
#define MAX_COMMANDS 10
#define MAX_ARGS 20

typedef struct {
    char command[MAX_INPUT];
    char *args[MAX_ARGS];
    int argc;

    char input_file[100];
    char output_file[100];
    char error_file[100];

    int has_input;
    int has_output;
    int has_error;
} Command;

void trim_spaces(char *str)
{
    char *start = str;

    while (*start == ' ' || *start == '\t')
        start++;

    if (start != str)
        memmove(str, start, strlen(start) + 1);

    int len = strlen(str);

    while (len > 0 &&
           (str[len - 1] == ' ' || str[len - 1] == '\t'))
        len--;
}

```

```

int parse_command(char *text, Command *cmd)
{
    memset(cmd, 0, sizeof(Command));

    trim_spaces(text);

    if (strlen(text) == 0)
        return 0;

    strcpy(cmd->command, text);

    char temp[MAX_INPUT];
    strcpy(temp, text);

    /* Parse input redirection */
    char *pos = strstr(temp, "<");

    if (pos != NULL)
    {
        *pos = '\0';

        char *file = pos + 1;

        while (*file == ' ')
            file++;

        if (*file == '\0')
        {
            printf("Error: Input file missing.\n");
            return 0;
        }

        sscanf(file, "%99s", cmd->input_file);
        cmd->has_input = 1;
    }

    /* Parse error redirection */
    pos = strstr(temp, ">");
}

```

```

        dup2(fd, STDERR_FILENO);
        close(fd);
    }

    /* Close pipe descriptors */
    for (int j = 0; j < command_count - 1; j++)
    {
        close(pipes[j][0]);
        close(pipes[j][1]);
    }

    execvp(commands[i].args[0],
            commands[i].args);

    perror("execvp");
    exit(1);
}

/* Parent closes pipe descriptors */
for (int i = 0; i < command_count - 1; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* Wait for all children */
for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nAll pipeline processes completed.\n");
printf("Execution order validated.\n");
printf("Resources cleaned successfully.\n");
printf("Complex pipeline test completed.\n");

return 0;
}

```

```

root@Shamikahasini:~# mkdir -p ~/week8/task2
root@Shamikahasini:~# cd ~/week8/task2
root@Shamikahasini:~/week8/task2# pwd
/root/week8/task2
root@Shamikahasini:~/week8/task2# nano complex_pipeline.c
root@Shamikahasini:~/week8/task2# gcc complex_pipeline.c -o T2
root@Shamikahasini:~/week8/task2# ls -l
total 32
-rwxr-xr-x 1 root root 21224 Sep 24 09:34 T2
-rw-r--r-- 1 root root 7587 Sep 24 09:34 complex_pipeline.c
root@Shamikahasini:~/week8/task2# echo "Hello Linux" > input.txt
root@Shamikahasini:~/week8/task2# cat input.txt
Hello Linux
root@Shamikahasini:~/week8/task2# ./T2
=====
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections
=====

Enter command: cat input.txt

=== Execution Plan ===
Stage 1: cat input.txt

Executing...
Hello Linux

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
root@Shamikahasini:~/week8/task2# ./T2
=====
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections
=====

Enter command: cat input.txt | grep Linux | wc -w

=== Execution Plan ===
Stage 1: cat input.txt

Executing...
Hello Linux

```

```

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
root@Shamikahasini:~/week8/task2# ./T2
=====
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections
=====

Enter command: cat input.txt > result.txt

=== Execution Plan ===
Stage 1: cat input.txt > result.txt
Output : result.txt

Executing...

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
root@Shamikahasini:~/week8/task2# cat result.txt
Hello Linux
root@Shamikahasini:~/week8/task2# ./T2
=====
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections
=====

Enter command: cat input.txt | grep Linux > result.txt

=== Execution Plan ===
Stage 1: cat input.txt

Executing...
Hello Linux

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
root@Shamikahasini:~/week8/task2# cat result.txt

```

```

=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections
=====

Enter command: wc -w < data.txt

=== Execution Plan ===
Stage 1: wc -w < data.txt
Input : data.txt

Executing...
3

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
root@Shamikahasini:~/week8/task2# ./T2
=====
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections
=====

Enter command: cat missing.txt 2> error.txt

=== Execution Plan ===
Stage 1: cat missing.txt 2> error.txt
Error : error.txt

Executing...

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
root@Shamikahasini:~/week8/task2# cat error.txt
cat: missing.txt: No such file or directory
root@Shamikahasini:~/week8/task2# ./T2
=====
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections
=====

Enter command: cat input.txt >

```

```

=====
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections
=====

Enter command: cat input.txt | | wc -w

=== Execution Plan ===
Stage 1: cat input.txt

Executing...
Hello Linux

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
root@Shamikahasini:~/week8/task2# ./T2
=====
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections
=====

Enter command: invalidcommand

=== Execution Plan ===
Stage 1: invalidcommand

Executing...
execvp: No such file or directory

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
root@Shamikahasini:~/week8/task2# ls -lh
total 48K
-rwxr-xr-x 1 root root 21K Sep 24 09:34 T2
-rw-r--r-- 1 root root 7.5K Sep 24 09:34 complex_pipeline.c
-rw-r--r-- 1 root root 19 Sep 24 09:36 data.txt
-rw-r--r-- 1 root root 44 Sep 24 09:37 error.txt
-rw-r--r-- 1 root root 12 Sep 24 09:34 input.txt
-rw-r--r-- 1 root root 12 Sep 24 09:36 result.txt
root@Shamikahasini:~/week8/task2#

```

Observation

The program successfully parsed complex commands containing multiple pipeline stages and redirection operators. It generated an execution plan showing the order of commands and their input, output, and error redirections before execution. Pipes were created between consecutive processes, while file descriptors were duplicated to implement input, output, and stderr redirection. Multiple processes were launched and synchronized using `waitpid()`. Different test scenarios, including multiple pipelines, input and output redirection, error redirection, missing files, and invalid pipeline syntax, were used to verify the correct execution and handling of complex commands.

Session-16

Task-1

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 100
#define MAX_CMD 256

typedef struct {
    int job_id;
    pid_t pid;
    char command[MAX_CMD];
    int active;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void check_jobs() {
    int status;

    for (int i = 0; i < job_count; i++) {
        if (jobs[i].active) {
            pid_t result = waitpid(jobs[i].pid, &status, WNOHANG);

            if (result > 0) {
                jobs[i].active = 0;

                if (WIFEXITED(status))
                    printf("\n[%d] Done      %s\n",
                        jobs[i].job_id, jobs[i].command);
                else
                    printf("\n[%d] Terminated  %s\n",
                        jobs[i].job_id, jobs[i].command);
            }
        }
    }
}

char command[MAX_CMD];
strcpy(command, input);

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    continue;
}

if (pid == 0) {
    execl("/bin/sh", "sh", "-c", input, NULL);

    perror("exec");
    exit(1);
}

if (background) {
    if (job_count < MAX_JOBS) {
        jobs[job_count].job_id = job_count + 1;
        jobs[job_count].pid = pid;
        strcpy(jobs[job_count].command, command);
        jobs[job_count].active = 1;

        printf("[%d] %d\n",
            jobs[job_count].job_id,
            pid);

        job_count++;
    }

    printf("Background process started. Prompt returned immediately.\n");
} else {
    waitpid(pid, NULL, 0);
}

return 0;
}
```

```

root@Shamikahasini:~# mkdir background_jobs
cd background_jobs
root@Shamikahasini:~/background_jobs# nano bgjobs.c
root@Shamikahasini:~/background_jobs# gcc bgjobs.c -o bgjobs
root@Shamikahasini:~/background_jobs# ls
bgjobs  bgjobs.c
root@Shamikahasini:~/background_jobs# ./bgjobs
myshell> sleep 5
myshell> sleep 10 &
[1] 626
Background process started. Prompt returned immediately.
myshell> jobs

[1] Done      sleep 10

Background Jobs:
No active background jobs.
myshell> sleep 5 &
[2] 630
Background process started. Prompt returned immediately.
myshell> sleep 10 &
[3] 632
Background process started. Prompt returned immediately.

[2] Done      sleep 5
myshell> sleep 15 &
[4] 634
Background process started. Prompt returned immediately.
myshell> jobs

[3] Done      sleep 10

Background Jobs:
[4] PID=634 RUNNING sleep 15
myshell> jobs

[4] Done      sleep 15

Background Jobs:
No active background jobs.
myshell> echo "Hello Background" &
[5] 638
Background process started. Prompt returned immediately.
myshell> Hello Background

```

Observation: The background process was successfully detected and launched without blocking the shell. The prompt returned immediately after starting the process. The job ID and PID were stored, and the process was monitored using `waitpid()` with `WNOHANG`. After completion, the job was reported as **Done**, and no active background jobs remained.

Task-2

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 100

typedef enum {
    READY,
    RUNNING,
    COMPLETED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[100];
    JobState state;
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;

const char *state_name(JobState state) {
    switch (state) {
        case READY:
            return "READY";
        case RUNNING:
            return "RUNNING";
        case COMPLETED:
            return "COMPLETED";
        default:
            return "UNKNOWN";
    }
}

void add_job(pid_t pid, pid_t pgid, char *command) {
    if (job_count >= MAX_JOBS) {

void remove_completed_jobs() {
    for (int i = 0; i < job_count; ) {
        if (job_table[i].state == COMPLETED) {
            printf("Removing completed job [%d] PID=%d\n",
                job_table[i].job_id,
                job_table[i].pid);

            for (int j = i; j < job_count - 1; j++) {
                job_table[j] = job_table[j + 1];
            }
            job_count--;
        } else {
            i++;
        }
    }
}

void display_jobs() {
    update_jobs();

    printf("\n===== JOB TABLE =====\n");

    if (job_count == 0) {
        printf("No jobs in table.\n");
        return;
    }

    printf("%-8s %-8s %-8s %-15s %-12s\n",
        "Job ID",
        "PID",
        "PGID",
        "Command",
        "State");
}
```

```

GNU nano 7.2 jobtable.c *
    display_jobs();
    continue;
}

if (strcmp(command, "clean") == 0) {
    update_jobs();
    remove_completed_jobs();
    continue;
}

if (strlen(command) == 0)
    continue;

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
    continue;
}

if (pid == 0) {
    setpgid(0, 0);

    execl("/bin/sh", "sh", "-c", command, NULL);

    perror("execl");
    exit(1);
}

setpgid(pid, pid);
add_job(pid, pid, command);
printf("Job added: PID=%d PGID=%d\n", pid, pid);
}
return 0;
}

Job added: PID=742 PGID=742
jobtable> sleep 20
Job added: PID=746 PGID=746

jobtable> jobs

===== JOB TABLE =====
Job ID  PID    PGID    Command    State
-----
1       742     742     sleep 20    COMPLETED
2       746     746     sleep 20    RUNNING

jobtable> sleep 10
Job added: PID=748 PGID=748

jobtable> sleep 20
Job added: PID=754 PGID=754

jobtable> sleep 30
Job added: PID=756 PGID=756

jobtable> jobs

===== JOB TABLE =====
Job ID  PID    PGID    Command    State
-----
1       742     742     sleep 20    COMPLETED
2       746     746     sleep 20    COMPLETED
3       748     748     sleep 10    COMPLETED
4       754     754     sleep 20    RUNNING
5       756     756     sleep 30    RUNNING

jobtable> clean
Removing completed job [1] PID=742
Removing completed job [2] PID=746
Removing completed job [3] PID=748
Removing completed job [4] PID=754
Removing completed job [5] PID=756

jobtable> jobs

===== JOB TABLE =====

```

Observation:

The job table successfully stores process information including job ID, PID, PGID, command and state. Running processes are tracked and their states are updated after completion. Completed jobs can be removed from the table using the clean operation, validating proper job-record management.

